

Guia de Revisão — EX188 / DO180

Podman e Containers — Perguntas e respostas para estudo

Tarefa 1

PERGUNTA

Instale o podman na máquina workstation e adicione registry:5000 como um registro inseguro (insecure registry). Também garanta que o podman pesquise nesse registro antes de tentar qualquer outro.

RESPOSTA

Instale o pacote e edite o arquivo de configuração de registries:

```
sudo dnf install -y podman

# /etc/containers/registries.conf
unqualified-search-registries = ["registry:5000", "registry.access.redhat.com", "docker.io"]

[[registry]]
location = "registry:5000"
insecure = true
```

Colocar registry:5000 primeiro em unqualified-search-registries garante que ele seja pesquisado antes dos demais registros.

Tarefa 2

PERGUNTA

Baixe (pull) a imagem mais recente do httpd (latest) do registro registry.do180.lab e execute-a como root em segundo plano com os seguintes parâmetros:

- Publique a porta 80 do container na porta 8080 do host
- Anexe o diretório /web como volume mapeado para /usr/local/apache2/htdocs
- Dê ao container o nome reg-httpd
- Faça o container iniciar como um serviço chamado reg-httpd.service

RESPOSTA

```
sudo podman pull registry.do180.lab/httpd:latest

sudo podman run -d --name reg-httpd \
-p 8080:80 \
-v /web:/usr/local/apache2/htdocs:Z \
registry.do180.lab/httpd:latest

cd /etc/systemd/system
sudo podman generate systemd --new --name reg-httpd --files
sudo systemctl daemon-reload
```

```
sudo systemctl enable --now reg-httpd.service
```

Tarefa 3

PERGUNTA

Baixe a versão mais recente da imagem mariadb do registro em registry.do180.lab. Execute um container em segundo plano com os seguintes parâmetros:

- Dê ao container o nome testql
- Publique a porta 3306
- Defina as variáveis de ambiente: MYSQL_USER=duffman, MYSQL_PASSWORD=saysoyeah, MYSQL_ROOT_PASSWORD=SQLp4ss, MYSQL_DATABASE=beer

Conecte-se como root e verifique a existência do banco beer, e execute /sql/beer.sql para inserir os dados.

RESPOSTA

```
podman pull registry.do180.lab/mariadb:latest

podman run -d --name testql \
-p 3306:3306 \
-e MYSQL_USER=duffman \
-e MYSQL_PASSWORD=saysoyeah \
-e MYSQL_ROOT_PASSWORD=SQLp4ss \
-e MYSQL_DATABASE=beer \
registry.do180.lab/mariadb:latest

echo "show databases;" | mysql -u-duffman -h 192.168.88.4 -psaysoyeah
mysql -uroot -h 192.168.88.4 -pSQLp4ss < /sql/beer.sql
```

Tarefa 4

PERGUNTA

Use o skopeo para encontrar todas as tags associadas ao httpd no servidor registry.do180.lab. Crie e execute um container com os seguintes parâmetros:

- Use uma imagem httpd que NÃO esteja marcada com latest
- Publique a porta 80 na porta 8008
- Dê ao container o nome alp-httpd
- Execute o container em modo destacado (detached)

RESPOSTA

```
skopeo list-tags docker://registry.do180.lab/httpd

# escolha uma tag diferente de "latest", ex: 2.4
podman run -d --name alp-httpd -p 8008:80 registry.do180.lab/httpd:2.4
```

Tarefa 5

PERGUNTA

Faça o seguinte com o container alp-httpd recém-criado:

- Execute um shell interativo (/bin/sh) no container
- Altere "It works!" para "It Twerks!" em index.html
- Use curl para testar suas alterações
- Faça commit para registry:5000/httpd:twerks
- Envie (push) a imagem para o servidor de registro
- Encerre alp-httpd e crie tw-httpd baseado na imagem twerks (porta 8008)

RESPOSTA

```
podman exec -it alp-httpd /bin/sh
# dentro do container:
sed -i 's/It works!/It Twerks!/' /usr/local/apache2/htdocs/index.html
exit

curl http://localhost:8008

podman commit alp-httpd registry:5000/httpd:twerks
podman push registry:5000/httpd:twerks

podman kill alp-httpd
podman run -d --name tw-httpd -p 8008:80 registry:5000/httpd:twerks
```

Tarefa 6

PERGUNTA

Complete /dockerfiles/nginx/Dockerfile para criar uma imagem que:

- Parta de almalinux:9
- Instale o nginx
- Publique a porta 80
- Copie index.html e duffman.png para /usr/share/nginx/html
- Execute nginx -g daemon off

Marque como quay.io/SEUUSUARIO/duff-nginx:1.0, faça push, e execute um container 'duffman' publicando 8989:80. Teste com curl.

RESPOSTA

```
# /dockerfiles/nginx/Dockerfile
FROM almalinux:9
RUN dnf install -y nginx && dnf clean all
COPY index.html duffman.png /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

```
podman build -t quay.io/SEUUSUARIO/duff-nginx:1.0 \
-f /dockerfiles/nginx/Dockerfile /dockerfiles/nginx
podman login quay.io
podman push quay.io/SEUUSUARIO/duff-nginx:1.0

podman run -d --name duffman -p 8989:80 quay.io/SEUUSUARIO/duff-nginx:1.0
curl localhost:8989
```

Tarefa 7

PERGUNTA

Complete /dockerfiles/mosquitto/Dockerfile para criar uma imagem que:

- Use almalinux:9, com info de maintainer
- Crie o usuário duffman
- Instale epel-release e depois mosquitto
- Crie /colors a partir de colors.tar
- Mova skeeter.sh para /skeeter.sh e o torne executável
- Exponha a porta 1883
- Mude para o usuário duffman e execute skeeter.sh

Build com tag skeeter:1.0. Execute mosquitto-1 publicando 11883:1883 e teste com mosquitto_sub.

RESPOSTA

```
# /dockerfiles/mosquitto/Dockerfile
FROM almalinux:9
LABEL maintainer="Duffman <duffman@duff.com>"
RUN useradd duffman
RUN dnf install -y epel-release && dnf install -y mosquitto
ADD colors.tar /colors
COPY skeeter.sh /skeeter.sh
RUN chmod +x /skeeter.sh
EXPOSE 1883
USER duffman
CMD ["/skeeter.sh"]

podman build -t skeeter:1.0 -f /dockerfiles/mosquitto/Dockerfile /dockerfiles/mosquitto
podman run -d --name mosquitto-1 -p 11883:1883 skeeter:1.0
mosquitto_sub -h 127.0.0.1 -p 11883 -t "#"
```

Tarefa 8

PERGUNTA

Baseado na tarefa 3: crie quatro Podman secrets (mysql_user, mysql_password, mysql_root_password, mysql_database) a partir de arquivos em ~/. Execute um container secretsdb (imagem mariadb, porta 3306 publicada em 3307) usando esses secrets como variáveis de ambiente. Depois comprove o funcionamento com queries.

RESPOSTA

```
echo -n "duffman" > ~/mysql_user
echo -n "saysoyeah" > ~/mysql_password
echo -n "SQLp4ss" > ~/mysql_root_password
echo -n "beer" > ~/mysql_database

podman secret create mysql_user ~/mysql_user
podman secret create mysql_password ~/mysql_password
podman secret create mysql_root_password ~/mysql_root_password
podman secret create mysql_database ~/mysql_database

podman run -d --name secretsdb -p 3307:3306 \
--secret=mysql_user,type=env,target=MYSQL_USER \
--secret=mysql_password,type=env,target=MYSQL_PASSWORD \
--secret=mysql_root_password,type=env,target=MYSQL_ROOT_PASSWORD \
--secret=mysql_database,type=env,target=MYSQL_DATABASE \
registry.do180.lab/mariadb:latest

echo "show databases;" | mysql -uduffman -psaysoyeah -h workstation -P 3307
mysql -uroot -pSQLp4ss -h workstation -P 3307 < /sql/beer.sql
echo 'select * from types' | mysql -uduffman -psaysoyeah -h workstation -P 3307 beer
```

Tarefa 9

PERGUNTA

Crie ~/task9.dockerfile baseado em almalinux:9 que crie um usuário (padrão joe, substituível por ARG no build) e execute whoami. Construa hello-joe:1.0 e hello-lisa:1.0; ao rodar, devem exibir "joe" e "lisa" respectivamente. Lembre-se: ARG só vale no build, não no runtime.

RESPOSTA

```
# ~/task9.dockerfile
FROM almalinux:9
ARG USERNAME=joe
RUN useradd $USERNAME
USER $USERNAME
CMD ["whoami"]

podman build -t hello-joe:1.0 -f ~/task9.dockerfile .
podman build --build-arg USERNAME=lisa -t hello-lisa:1.0 -f ~/task9.dockerfile .

podman run --rm hello-joe:1.0 # -> joe
podman run --rm hello-lisa:1.0 # -> lisa
```

Tarefa 10

PERGUNTA

Baseado no Voting App do Docker Samples.

Complete /dockerfiles/voting/docker-compose.yml com: redes front-end (bridge) e back-end (internal); volume db-data; serviços redis, db (postgres:9.4 + volume + env vars), vote (porta 5000, depende de redis), result (porta 5001, depende de db) e worker (depende de db e redis).

RESPOSTA

```
# /dockerfiles/voting/docker-compose.yml
version: "3"

networks:
  front-end:
  driver: bridge
  back-end:
  driver: internal

volumes:
  db-data:

services:
  redis:
    image: redis
    container_name: redis
    networks:
      - front-end

  db:
    image: postgres:9.4
    container_name: db
    networks:
      - back-end
    volumes:
      - db-data:/var/postgres
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres

  vote:
    image: dockersamples/examplevotingapp_vote
    container_name: vote
    networks:
      - back-end
      - front-end
    ports:
      - "5000:80"
    depends_on:
      - redis

  result:
    image: dockersamples/examplevotingapp_result
    container_name: result
    networks:
      - back-end
      - front-end
    ports:
      - "5001:80"
```

```
depends_on:
- db

worker:
image: dockersamples/examplevotingapp_worker
container_name: worker
networks:
- back-end
- front-end
depends_on:
- db
- redis

podman-compose -f /dockerfiles/voting/docker-compose.yml up -d
```

Tarefa 11

PERGUNTA

Monte (mount) a imagem almalinux:9 no sistema de arquivos do host, SEM executar o container. Salve /etc/os-release em ~/task11.txt.

RESPOSTA

```
ctr=$(podman create almalinux:9)
mnt=$(podman mount $ctr)
cat $mnt/etc/os-release > ~/task11.txt
podman umount $ctr
podman rm $ctr
```

Tarefa 12

PERGUNTA

Crie ~/mywebsite/index.html com "Welcome to my website." Baixe o httpd oficial, descubra o document root, e execute um container "mywebsite" (porta 8888) fazendo bind mount de ~/mywebsite/ nesse diretório. Comprove que o site funciona.

RESPOSTA

```
mkdir -p ~/mywebsite
echo "Welcome to my website." > ~/mywebsite/index.html

podman pull docker.io/library/httpd
podman run --rm docker.io/library/httpd \
cat /usr/local/apache2/conf/httpd.conf | grep DocumentRoot
# DocumentRoot "/usr/local/apache2/htdocs"

podman run -d --name mywebsite -p 8888:80 \
-v ~/mywebsite:/usr/local/apache2/htdocs:Z \
docker.io/library/httpd
```

```
curl http://localhost:8888
```

Tarefa 13

PERGUNTA

Execute um container "runsalways" (httpd oficial, porta 9999) que sempre reinicie, mesmo após reboot, SEM criar um serviço systemd próprio. Comprove o funcionamento e o reinício.

RESPOSTA

```
podman pull docker.io/library/httpd
podman run -d --name runsalways --restart=always -p 9999:80 docker.io/library/httpd

curl http://localhost:9999
podman kill runsalways
sleep 3
podman ps # confirma reinício automático

# unit já fornecido pelo pacote podman (não é criado por você):
sudo systemctl enable --now podman-restart.service
```

Dica: como o unit podman-restart.service já vem com o pacote podman, habilitá-lo satisfaz o requisito de não criar seu próprio serviço.

Tarefa 14

PERGUNTA

Copie /echo para ~/echo. Encontre a tag slim baseada em Debian Trixie da imagem python. Crie um Dockerfile que rode python3 (com argumento) como usuário app (uid 1000). Ajuste permissões, ownership e contexto SELinux MANUALMENTE (sem :Z). Execute o container 'echo' (UDP 5500->5000, bind mount ~/echo:/app) e teste via UDP.

RESPOSTA

```
skopeo list-tags docker://docker.io/library/python | grep -i "slim-trixie"
# ex.: "3.13-slim-trixie"
podman pull docker.io/library/python:3.13-slim-trixie

cp -r /echo ~/echo

# ~/echo/Dockerfile
FROM python:3.13-slim-trixie
RUN useradd -u 1000 app
USER app
ENTRYPOINT ["python3"]

podman build -t echo:1.0 -f ~/echo/Dockerfile ~/echo

# Ajuste manual (sem usar :Z no volume)
chown -R 1000:1000 ~/echo
```

```
chmod -R u+rX ~/echo
sudo semanage fcontext -a -t container_file_t "$HOME/echo(/.*)?"
sudo restorecon -Rv ~/echo

podman run -d --name echo \
-p 5500:5000/udp \
-v ~/echo:/app \
echo:1.0 /app/udp_echo_server.py

echo "teste" > /dev/udp/workstation/5500
podman logs echo
```

Tarefa 15

PERGUNTA

Não faz parte dos objetivos do exame.

Copie /peapod para ~/peapod/ (pop.sh executável). Crie o pod "peapod" (porta 7777, bind mount ~/peapod em /usr/local/apache2/htdocs). Adicione um container httpd e um container almalinux:9 que roda pop.sh. Inicie o pod e valide que a página muda a cada 10s.

RESPOSTA

```
cp -r /peapod ~/peapod
chmod +x ~/peapod/pop.sh

podman pod create --name peapod -p 7777:80

podman run -d --pod peapod --name httpd-peapod \
-v ~/peapod:/usr/local/apache2/htdocs:Z \
docker.io/library/httpd

podman run -d --pod peapod --name almalinux-peapod \
-v ~/peapod:/usr/local/apache2/htdocs:Z \
docker.io/library/almalinux:9 \
/usr/local/apache2/htdocs/pop.sh

curl http://workstation:7777
```

Tarefa 16

PERGUNTA

Não faz parte dos objetivos do exame.

Crie um serviço systemd de usuário via Quadlet (não podman generate) que rode uptime-kuma (porta 3001), com volume kuma-data em /app/data, nome 'kuma', sempre reiniciando e iniciando no boot.

RESPOSTA

```
mkdir -p ~/.config/containers/systemd

# ~/.config/containers/systemd/kuma.container
[Unit]
Description=Uptime Kuma container

[Container]
Image=docker.io/louislam/uptime-kuma
ContainerName=kuma
PublishPort=3001:3001
Volume=kuma-data.volume:/app/data

[Service]
Restart=always

[Install]
WantedBy=default.target

# ~/.config/containers/systemd/kuma-data.volume
[Volume]

systemctl --user daemon-reload
systemctl --user enable --now kuma.service
loginctl enable-linger $USER
```

Validar em <http://workstation:3001/dashboard>